

BÁO CÁO PHASE 7
THÔNG BÁO TRONG ỨNG DỤNG

Dự án: Social Networking Promax

Sinh viên: Đường Gia Bằng

Mã sinh viên: 22026537.....

Lớp: K67I-IT20.....

Giảng viên hướng dẫn: TS. Nghiêm Nguyễn Việt Dũng.....

Thời gian: 19/07/2026

1. Mục tiêu của Phase 7

Sau Phase 6, hệ thống đã có nền tảng Socket.IO cho chat và gọi video. Phase 7 tiếp tục dùng nền tảng đó để xây dựng thông báo trong ứng dụng. Mục tiêu là giúp người dùng không bỏ lỡ tương tác quan trọng; thông báo được lưu trong cơ sở dữ liệu, có trạng thái đã đọc/chưa đọc và vẫn xem lại được sau khi kết nối lại.

Các mục tiêu chính của Phase 7 gồm:

- Tạo bảng Notifications, model và service trung tâm để lưu, truy vấn, cập nhật thông báo.
- Tạo sáu loại thông báo: cảm xúc, bình luận, lời mời/chấp nhận kết bạn, chia sẻ và tin nhắn ngoại tuyến.
- Cung cấp API danh sách có con trỏ, giới hạn và bộ lọc tất cả/chưa đọc/đã đọc.
- Đánh dấu một hoặc toàn bộ thông báo đã đọc và đồng bộ số lượng chưa đọc.
- Phát new_notification và notification_count tới room riêng bằng Socket.IO.
- Hoàn thiện NotificationBell, badge, dropdown, toast và trang /notifications.
- Tạo API lưu token FCM; push nền chỉ triển khai khi có cấu hình Firebase thật.

2. Kết quả đã hoàn thành

Phase 7 đã hoàn thành phần bắt buộc là thông báo trong ứng dụng. Bản ghi được lưu trong PostgreSQL, các sự kiện chính có thể tạo thông báo và giao diện cập nhật qua Socket.IO. Notification là tác vụ bổ sung nên lỗi phát socket không làm hỏng hành động gốc.

Về backend, dự án đã hoàn thành các phần sau:

- Tạo migration/model Notification và các chỉ mục theo người dùng, trạng thái, thời gian, loại và referenceId.
- Tạo service xử lý create, list/filter/cursor, unread count, mark read/read all và bỏ qua self-event.
- Tạo ba API list, read, read-all; tất cả qua xác thực, validation và giới hạn theo chủ sở hữu.
- Tích hợp like, comment, share, friend_request và friend_accept vào các service nghiệp vụ.
- Tạo thông báo message cho thành viên ngoại tuyến; emit new_notification và notification_count qua room user: {id}.

Về frontend, dự án đã hoàn thành các phần sau:

- Tạo API client và Redux slice quản lý danh sách, unread count, phân trang, tải, lỗi và toast.
- Thêm NotificationBell với badge 99+, tám item gần nhất, trạng thái rỗng, read-all và toast thời gian thực.
- Tạo trang /notifications có bộ lọc Tất cả/Chưa đọc, tải thêm và thao tác đọc từng item.
- Điều hướng tới bài viết, bạn bè, hồ sơ hoặc cuộc hội thoại theo loại thông báo.
- Bổ sung POST /api/users/fcm-token và hàm client lưu token có validation. Firebase Admin SDK, service worker, đăng ký token tự động từ trình duyệt và push nền chưa được cấu hình nên FCM mới ở mức nền tảng.

3. Những kiến thức đã học được

Phase 7 cho thấy một hệ thống thông báo không chỉ là biểu tượng chuông và một con số màu đỏ. Chức năng này nối nhiều miền nghiệp vụ với nhau, phải lưu dữ liệu bền vững, cập nhật thời gian thực, giữ đúng quyền truy cập và vẫn xử lý được khi người dùng đang ngoại tuyến hoặc socket tạm mất kết nối.

3.1. Hiểu vai trò của một notification service trung tâm

Nếu mỗi service tự tạo nội dung, tự đếm chưa đọc và tự phát socket thì logic sẽ nhanh chóng bị lặp. Việc gom createNotification, listNotifications, markRead và markAllRead vào một service giúp các miền post, comment, friend và chat chỉ cần gửi đúng người nhận, loại và referenceId.

Service trung tâm cũng là nơi áp dụng quy tắc chung như bỏ qua thông báo cho chính mình, chuẩn hóa dữ liệu người tạo sự kiện và nuốt lỗi phát socket để tương tác gốc vẫn thành công. Cách tách này làm ranh giới trách nhiệm rõ hơn và thuận lợi cho kiểm thử.

3.2. Học cách thiết kế dữ liệu và phân trang cho thông báo

Thông báo cần được sắp xếp mới nhất trước và thường được đọc theo từng trang nhỏ. Phase 7 dùng createdAt làm con trỏ, lấy thêm một bản ghi để xác định hasMore và trả nextCursor cho lần tải tiếp theo. Cách này phù hợp hơn offset khi dữ liệu mới liên tục xuất hiện.

Các chỉ mục userId-createdAt và userId-isRead-createdAt giúp hai truy vấn phổ biến là danh sách mới nhất và số lượng chưa đọc. Mỗi bản ghi còn có type và referenceId để giao diện biết nội dung nào cần mở khi người dùng nhấp vào thông báo.

Em cũng hiểu rằng con trỏ chỉ theo thời gian vẫn có trường hợp biên khi nhiều bản ghi cùng timestamp. Bản thiết kế hiện tại đủ cho quy mô dự án, nhưng khi dữ liệu lớn hơn nên dùng con trỏ ghép createdAt và id để tránh trùng hoặc bỏ sót.

3.3. Kết hợp dữ liệu lưu trữ với cập nhật thời gian thực

Socket.IO giúp chuông, badge và toast thay đổi ngay khi sự kiện xảy ra, nhưng socket không thay thế cơ sở dữ liệu. Notification vẫn được lưu trước, sau đó server mới phát new_notification và notification_count. Nếu người nhận ngoại tuyến, bản ghi còn nguyên để tải lại khi đăng nhập.

Khi socket kết nối hoặc kết nối lại, Navbar gọi lại API danh sách gần nhất. Cách kết hợp này giúp giao diện nhanh khi kết nối tốt nhưng vẫn có nguồn dữ liệu đáng tin cậy nếu event bị bỏ lỡ.

3.4. Học cách tích hợp sự kiện qua nhiều miền nghiệp vụ

Một thông báo like khác thông báo kết bạn hoặc tin nhắn về thời điểm phát sinh và referenceId. Like chỉ tạo khi có reaction mới, unlike không tạo; friend_accept gửi về người đã mời; message chỉ tạo cho thành viên không trực tuyến. Mỗi miền phải xác định đúng quy tắc trước khi gọi notification service.

Các lệnh tạo notification đều được gọi sau khi hành động chính đã thành công và có catch riêng. Điều này giúp một lỗi phụ ở notification không làm người dùng mất bình luận, chia sẻ hoặc lời mời kết bạn vừa thực hiện.

3.5. Học cách quản lý giao diện thông báo ở mức toàn ứng dụng

NotificationBell nằm trên Navbar nên có mặt ở mọi màn hình đã đăng nhập. Navbar lắng nghe new_notification để đưa bản ghi mới vào Redux và lắng nghe notification_count để đồng bộ badge. Toast dùng cùng dữ liệu thay vì tạo thêm một luồng trạng thái riêng.

Trang /notifications dùng lại Redux slice nhưng tải giới hạn lớn hơn, có bộ lọc và phân trang. Hàm merge theo id giúp tránh trùng khi vừa nhận sự kiện socket vừa tải thêm dữ liệu từ API.

3.6. Hiểu rõ hơn về quyền sở hữu và dữ liệu công khai

Mọi API notification đều đi qua requireAuth và service luôn gắn userId hiện tại vào điều kiện truy vấn. Khi người dùng cố đọc notification của người khác, hệ thống trả không tìm thấy thay vì tiết lộ bản ghi có tồn tại.

Dữ liệu fromUser chỉ được serialize qua bộ lọc publicUser, không trả mật khẩu, token hoặc thông tin nhạy cảm. Notification hỗ trợ trải nghiệm nhưng không được phép trở thành đường vòng vượt qua authorization của bài viết, hồ sơ hay cuộc hội thoại.

3.7. Học cách đánh giá đúng phạm vi kiểm thử

Unit test của notification service kiểm tra self-event, tạo và serialize bản ghi, lọc unread kèm unread count và thao tác mark read idempotent. Toàn bộ test backend hiện có cũng được chạy để phát hiện hồi quy ở auth, friends, posts, stories, chat và call.

Tuy vậy, unit test chưa chứng minh đầy đủ luồng hai trình duyệt, mất kết nối, hiển thị responsive hoặc quyền sở hữu qua API thật với database test. Các phần đó vẫn cần manual test hoặc E2E trước khi triển khai production.

3.8. Học cách ưu tiên Must-have và Stretch

Kế hoạch xác định thông báo trong ứng dụng là bắt buộc, còn FCM phụ thuộc Firebase project, Admin SDK và service worker. Phase 7 ưu tiên hoàn thành DB, API, Socket.IO và UI trước; API lưu fcmToken được chuẩn bị sẵn nhưng không tuyên bố push nền đã hoàn thành khi chưa có cấu hình thật.

4. Luồng hoạt động chính

- Người dùng thực hiện like, comment, kết bạn, chia sẻ hoặc gửi tin nhắn.
- Service nghiệp vụ kiểm tra quyền và hoàn tất thao tác chính trong database.
- Service gọi createNotification với người nhận, người gửi, type và referenceId.
- Notification service bỏ self-event, lưu isRead=false và nạp dữ liệu công khai.
- Server đếm unread rồi phát new_notification và notification_count tới room người nhận.
- Redux thêm bản ghi, cập nhật badge/toast; client tải lại danh sách khi reconnect.
- Khi nhấp item, client gọi mark read rồi điều hướng tới nội dung liên quan.
- Nếu ngoại tuyến, bản ghi vẫn nằm trong database; FCM thực tế chưa được gửi.

5. Khó khăn và cách xử lý

Khó khăn đầu tiên là tránh làm notification ảnh hưởng tới các chức năng đã ổn định. Việc tạo và phát thông báo được đặt sau thao tác chính, dùng catch riêng và service không ném lại lỗi socket. Nhờ vậy lỗi phụ không làm hỏng like, comment, friend hoặc message.

Khó khăn thứ hai là xác định đúng người nhận và ngăn self-event. Mỗi miền có quy tắc khác nhau, nhưng notification service vẫn kiểm tra lần cuối khi fromUserId trùng userId để tránh các bản ghi vô nghĩa.

Khó khăn thứ ba là giữ unread count nhất quán giữa API, Redux và Socket.IO. Server luôn đếm lại sau create/mark read, còn client chấp nhận notification_count làm giá trị chuẩn và tải lại dữ liệu khi socket reconnect.

Khó khăn cuối cùng là FCM cần hạ tầng ngoài dự án. Phase 7 chỉ lưu token có validation 10-4096 ký tự và ghi rõ giới hạn; Firebase Admin SDK, khóa dịch vụ và service worker không được thêm khi chưa có project/config chính thức.

6. Mức độ hoàn thiện

API với database thật, socket hai client, UI responsive, reference bị xóa và FCM background chưa được tự động hóa. Phase 7 đạt Must-have in-app; E2E, hiệu năng và push nền cần được phát triển tiếp ở Phase 8.

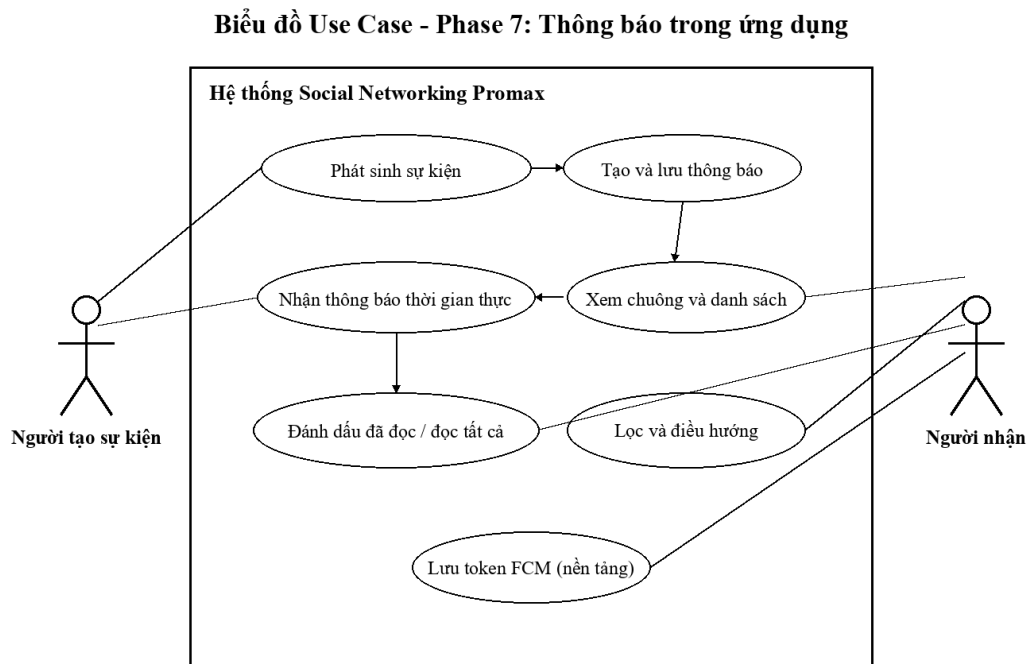
7. Hạn chế hiện tại và hướng phát triển

- Chưa có Firebase Admin SDK, service worker và cấu hình Firebase; hiện mới lưu fcmToken.
- Trạng thái online nằm trong memory một instance; nhiều server sẽ cần Redis/pub-sub.
- Chưa có cài đặt theo loại, gom nhóm, chống lặp, âm thanh, dọn bản ghi cũ hoặc email.
- Cursor chỉ dùng createdAt; nên ghép createdAt-id khi dữ liệu lớn.
- Lỗi socket được bỏ qua để bảo vệ luồng chính nhưng chưa có log, metric hoặc retry.

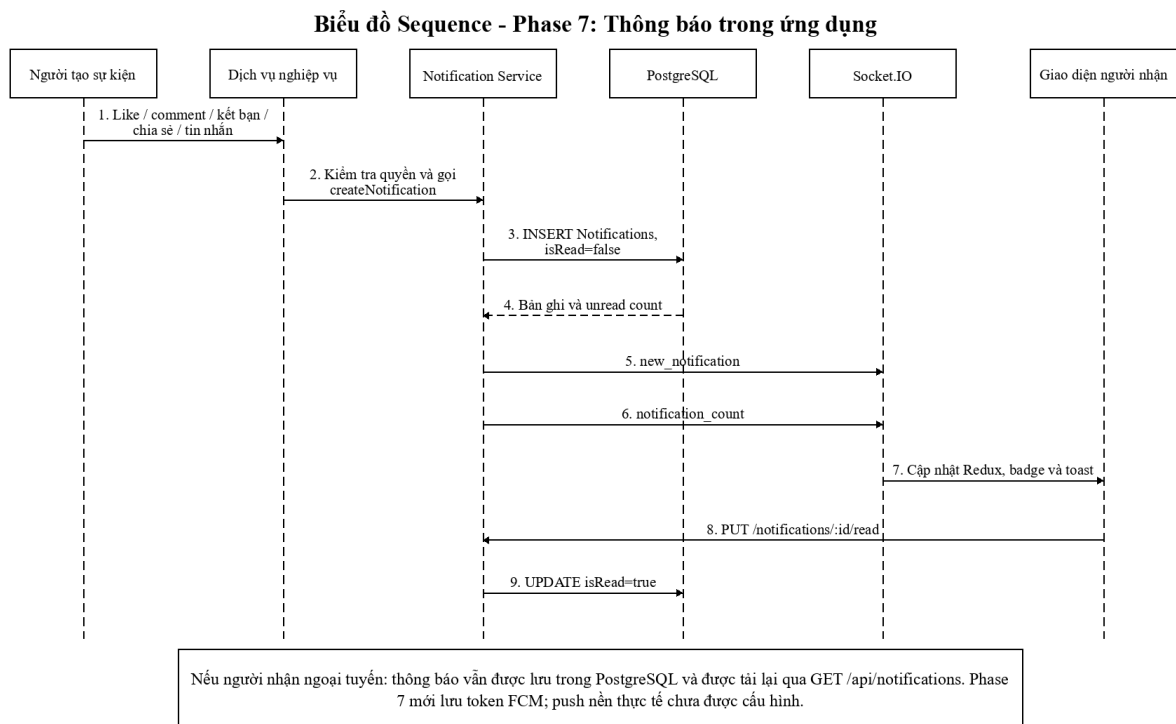
Bước tiếp theo là hoàn thiện FCM, cài đặt theo người dùng, gom nhóm, integration/E2E test và code splitting trong Phase 8.

8. Biểu đồ use case và sequence

Hai biểu đồ dưới đây mô tả phạm vi Phase 7. Biểu đồ use case thể hiện hành động của người tạo sự kiện và người nhận thông báo. Biểu đồ sequence mô tả thứ tự từ thao tác nghiệp vụ, lưu database, phát Socket.IO đến khi giao diện cập nhật và đánh dấu đã đọc.



Hình 1. Biểu đồ use case chức năng thông báo trong ứng dụng.



Hình 2. Biểu đồ sequence luồng tạo, nhận và đọc thông báo.

9. Kết luận

Phase 7 đã bổ sung cho Social Networking Promax một hệ thống thông báo trong ứng dụng tương đối đầy đủ: sáu loại sự kiện, lưu trữ PostgreSQL, danh sách có phân trang/lọc, trạng thái

đã đọc, unread count, cập nhật Socket.IO, NotificationBell, toast và trang quản lý riêng. Kết quả này giúp các chức năng post, friends và chat liên kết thành một trải nghiệm thống nhất hơn.

Điều quan trọng nhất em học được là thông báo phải vừa nhanh vừa đáng tin cậy. Socket.IO tạo cảm giác tức thời, còn database và API bảo đảm người dùng vẫn thấy dữ liệu khi ngoại tuyến hoặc kết nối lại. Phase 7 đã đạt phần Must-have; FCM thực tế, E2E, quan sát vận hành và tối ưu hiệu năng được giữ rõ ràng cho bước hoàn thiện tiếp theo.